

LINQ Range Method in C# with Examples

In this article, I will discuss the **LINQ Range Method in C#** with Examples. Please read our previous article discussing [How to Implement Paging using LINQ Skip and Take Methods in C#](#) with an Example. The LINQ Range Method belongs to the Generation Operator Category. So, let us first discuss what a Generation Operator is, and then we will see the Range method.

What is a Generation Operator?

As of now, we have discussed many LINQ Extension methods such as Select, Where, Any, etc. The Enumerable class also provides three Non-Extension Methods. They are as follows:

1. **Range()**
2. **Repeat<T>()**
3. **Empty<T>()**

These three methods allow us to create some specific type of Array, Sequence, or Collection using a single expression instead of creating them manually and populating them using some loop. That means these three methods return a new sequence or collection that implements the **IEnumerable<T>** interface.

Note: In this article, I will discuss the **LINQ Range Method in C#** with Examples. The other two methods, i.e., **Repeat** and **Empty**, will be discussed in our upcoming articles.

LINQ Range Method in C#:

The LINQ Range method is a static method of the Enumerable class that generates a sequence of integral numbers within a specified range. It is particularly useful when you need a simple sequence of numbers without manually initialing an array or list. The Range method takes two parameters: the start value and the count of sequential integers to generate.

The LINQ Range Method in C# generates a sequence of integral (integer) numbers within a specified range. The following is the signature of this method.

```
public static IEnumerable<int> Range(int start, int count);
```

This method takes 2 integer parameters. The first parameter (i.e., **int start**) specifies the value of the first integer in the sequence. The second parameter (i.e., **int count**) specifies the number of sequential integers to be generated. The return type of this method is **IEnumerable<int>**, which will contain a range of sequential integer numbers.

Note: When the count value is less than 0, or when the start + count – 1 value is larger than the System.Int32.MaxValue, then it will throw **ArgumentOutOfRangeException**.

Example to Understand LINQ Range Method in C#:

Let us see an example of the LINQ Range Method in C#. In the following example, we generate a sequence of integer numbers from 1 to 10 using the Range Method.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Generating Integer Numbers from 1 to 10
            IEnumerable<int> numberSequence = Enumerable.Range(1, 10);
            //Accessing the numberSequence using Foreach Loop
            foreach (int num in numberSequence)
            {
                Console.WriteLine($"{num} ");
            }
            Console.ReadKey();
        }
    }
}
```

Once you run the application, it will print the values from 1 to 10 as expected.

Example to Understand LINQ Range Method with Where Extension Method in C#:

Let us see an example to understand how to use the LINQ Range Method with the LINQ Where Extension Method. The following example prints all the even numbers which are present between 10 and 40. Here, first, we need to generate the sequence using the Range method, and then we need to filter the data using the Where extension method.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Range with Where Extension Method
            IEnumerable<int> EvenNumbers = Enumerable.Range(10, 40).Where(x =>
0);

            //Printing the Even Numbers between 10 and 40
            foreach (int num in EvenNumbers)
            {
                Console.WriteLine($"{num} ");
            }
        }
    }
}
```

```

        Console.ReadKey();
    }
}

```

Output:

```
10 12 14 16 18 20 22 24 26 28 30 32 34 36 38
```

Example to use Range Method with Select Extension Method in C#:

The following example shows how to generate a sequence of integers from 1 to 5 and then return the square of each number.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            IEnumerable<int> EvenNumbers = Enumerable.Range(1, 5).Select(x =>
                x * x);
            foreach (int num in EvenNumbers)
            {
                Console.WriteLine($"{num} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output: 1 4 9 16 25

Example to Understand LINQ Range Method with String Type

In the following example, first, we generate a sequence and pass the sequence to our CustomLogic method. Then, we merge the sequence with the return value from the CustomLogic method and return the result as `IEnumerable<string>`.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LinqDemo
{
    class Program
    {

```

```

static void Main(string[] args)
{
    IEnumerable<string> rangewithString = Enumerable.Range(1, 5).Select(
x) + " " + CustomLogic(x)).ToArray();
    foreach (var item in rangewithString)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
private static string CustomLogic(int x)
{
    string result = string.Empty;
    switch (x)
    {
        case 1:
            result = "1st";
            break;
        case 2:
            result = "2nd";
            break;
        case 3:
            result = "3rd";
            break;
        case 4:
            result = "4th";
            break;
        case 5:
            result = "5th";
            break;
    }
    return result;
}
}
}

```

Output:

```

1 1st
4 2nd
9 3rd
16 4th
25 5th

```

Note: This method is implemented by using deferred execution. The immediate return value is an object that stores all the information that is required to perform the action. The query represented by this method is not executed until the object is enumerated by calling its GetEnumerator method directly or using a for each loop.

When to Use the LINQ Range Method in C#?

The LINQ Range method is a convenient way to generate a sequence of integers, which can be particularly useful in a variety of scenarios:

- **Loops:** When you need a simple for-loop-like iteration in LINQ query syntax and don't want to use the traditional for or for each loop construct.
- **Generating Sequences:** Quickly create a sequence of numbers for testing or prototyping or when the exact numbers aren't important, just because you have a sequence.
- **Indexing:** When you need a sequence of indices to iterate over another collection or to perform an operation at each index.
- **Batch Processing:** For batch operations where you need to perform an action repeatedly a certain number of times.
- **Data Population:** When populating data structures with sample data, you might want a quick way to insert a range of numbers.
- **Scheduling:** In scheduling scenarios, you might generate a sequence representing days, weeks, or other time periods for further calculations.
- **Pagination:** To calculate page numbers for a paging system in a web or desktop application.

Here's an example of using Range in combination with Select to create a sequence of dates:

```
using System;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            DateTime startDate = new DateTime(2021, 1, 1);
            int daysToGenerate = 10;
            var dateSequence = Enumerable.Range(0, daysToGenerate)
                                         .Select(offset => startDate.AddDays(offset));
            foreach (var date in dateSequence)
            {
                Console.WriteLine(date.ToShortDateString());
            }

            Console.ReadKey();
        }
    }
}
```

In this example, Range generates a sequence of offsets, which are then transformed into a sequence of dates starting from a startDate.

It's important to understand that Range generates a sequence of memory integers, so working with a large range could lead to high memory consumption. Always ensure that the count of numbers you are generating is within the bounds of what your application can handle regarding performance and memory usage.

In the next article, I will discuss the [LINQ Repeat Method in C#](#) with Examples. Here, in this article, I try to explain the **LINQ Range Method in C#** with some different types of examples. I hope you enjoy this article.